

Trento Smart Mountain

Deliverable D3 – Running the Sprint!

Gruppo: ID – 6

Componenti: Federico Cattelan – 242111

Marco Christian Stoica – 246443

Giacomo Radin – 242907



Scadenza: 17/05/2026

Anno Accademico 2025/2026

“Le grandi cose non vengono fatte d’impulso, ma attraverso una serie di piccole cose combinate assieme.”



Abstract Sprint 1

Il Deliverable D3 descrive l’adozione del framework Agile SCRUM per lo sviluppo di Trento Smart Mountain. Vengono formalizzate le dinamiche del team durante lo *Sprint 1*, focalizzato sulla creazione delle fondamenta architetturali (Backend Node.js + MongoDB), dei servizi core di autenticazione JWT, della gestione completa delle sessioni escursione con tracking GPS, profilo altimetrico reale da GPX e integrazione meteo live (TINIA/meteo.report). Il documento dettaglia Product Backlog, strategia di branching, risultati degli Sprint meetings e i 3 bug critici identificati nell’audit interno di fine sprint – a dimostrazione di un processo Agile maturo basato sull’auto-critica continua.

Indice

1	Sezione Introduttiva	3
2	Sezione Generale	4
2.1	Strategia di Branching	4
2.2	Product Backlog	5
2.3	Definizione di “Done”	10
3	Sezione Sprint #1	11
3.1	Goal	11
3.2	Sprint Planning (Sprint Backlog)	11
3.3	Test Cases	18
3.4	Sprint Review	23
3.5	Product Backlog Refinement	24
3.6	Sprint Retrospective	24
A	- API implementate Sprint 1	26
B	Utilizzo Intelligenza Artificiale nello Sviluppo	28
C	Mockup delle Schermate	29
C.1	Schermate di Autenticazione e Registrazione	29
C.2	Schermate Home e Attività	31
C.3	Schermate di Gestione Sessione Escursione	33
C.4	Schermata di Tracciamento GPS	36

1 Sezione Introduttiva

Team Members

Nome	Cognome	Matricola	Account GitHub
Federico	Cattelan	242111	@federicoca
Marco Christian	Stoica	246443	@STUSSY-user
Giacomo	Radin	242907	@giacomoradin

Tabella 1: Componenti del team di sviluppo.

Tutti e tre i componenti hanno contribuito con commit al repository condiviso, come verificabile dalla cronologia GitHub.

Project Idea

Trento Smart Mountain (TSM) è un ecosistema digitale per l'escursionismo in Trentino-Alto Adige che integra **sicurezza attiva di gruppo** (tracciamento GPS in background, codici invito sessione, fallback SOS), **gamification educativa** (modello CAI di stima sforzo, punti per attività completate, futuri quiz NFC ai checkpoint di vetta) e **gestione rifugi** (account dedicati, telemetria IoT prevista). Il sistema combina un'app Android nativa (Kotlin 2.0 / Jetpack Compose, architettura MVVM, offline-first) con un backend Node.js + MongoDB (indici geospaziali 2dsphere, JWT) e un'integrazione meteo reale (TINIA / meteo.report) per supportare l'escursionista dalla pianificazione fino al ritorno a valle.

Links

- **Repository GitHub:** https://github.com/giacomoradin/Trento_Smart_Mountain
- **Swagger / API Docs:** <http://<host>:3000/api-docs> (file swagger-output.json incluso nel repository)
- **SwaggerHub (Alternativa Apiary):** <https://app.swaggerhub.com/apis/unitn-1d6/trento-smart-mountain-api/1.0.0>

2 Sezione Generale

2.1 Strategia di Branching

Il team ha adottato una strategia basata su *Git Flow* semplificata, evitando rigorosamente la logica “*Master only strategy*”.

Branch attivi e storici

Branch	Scopo	Stato
main	Release stabile; solo merge da PR approvate. Nessun push diretto.	● Attivo
UI	Branch di integrazione della UI dell'app mobile.	● Attivo
Develop	Branch di implementazione Sprint 1: convergenza tra feature mobile e API.	● Attivo
API-Meteo-Integration	Feature branch per l'integrazione meteo TINIA.	✓ Mergiato
auth-login-jwt	Feature branch per l'autenticazione JWT.	✓ Mergiato
crud-mongodb	Feature branch per le operazioni CRUD su MongoDB.	✓ Mergiato
Swagger-setup	Feature branch per la documentazione OpenAPI/Swagger.	✓ Mergiato
18-gestione-sessione-escursione	Feature branch relativo all'Issue #18 (gestione sessione escursione).	✓ Mergiato
Reorganizatio-Repo-Structure	Refactor della struttura delle cartelle del repository.	✓ Mergiato
bugfix/*	Branch dedicati alla risoluzione di bug critici (previsti per Sprint 2).	🕒 Pianificati

Tabella 2: Strategia di branching adottata dal team (GitFlow semplificato). I branch merged non vengono eliminati per consentire la verifica dei docenti.

Convenzioni operative

1. Una branch per ogni Issue GitHub (formato <numero>-<slug> oppure <feature>-<area>).
2. PR obbligatoria verso UI (Sprint 1) o main – mai push diretto su main.
3. Commit semantici: feat:, fix:, refactor:, docs:, chore:.
4. Merge tramite Pull Request con revisione esplicita di almeno un altro membro del team.

❗ Nota per i docenti: I branch **non vengono cancellati** dopo il merge, in conformità con le indicazioni del docente, per permettere la verifica completa della storia di sviluppo nel repository GitHub.

2.2 Product Backlog

Di seguito il Product Backlog in formato User Story agile (priorità MoSCoW). Lo stato riflette la situazione a fine Sprint 1 (16/05/2026).

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
3	Tutti gli utenti	Registrazione account	Come nuovo utente, voglio poter creare un account inserendo i miei dati, così da accedere alla piattaforma e usufruire dei suoi servizi.	1
1	Tutti gli utenti	Autenticazione al sistema	Come utente, voglio poter effettuare il login con le mie credenziali così da accedere alle funzionalità del sistema in base al mio ruolo.	2
4	Tutti gli utenti	Logout	Come utente autenticato voglio poter effettuare il logout dall'app, così da proteggere i miei dati su dispositivi condivisi.	3
2	Tutti gli utenti	Accesso offline con token JWT	Come utente già autenticato, voglio poter accedere all'app anche senza connessione internet, così da usare le funzionalità critiche in montagna.	5
18	Capogruppo	Gestione sessione escursione	Come Capogruppo, voglio poter creare una nuova sessione di escursione con i parametri del percorso, così da coordinare il gruppo e avviare il tracciamento.	8
6	Capogruppo	Aggiornamento sessione	Come capogruppo voglio poter aggiornare i parametri della sessione (nome, data, ora, partecipanti), così da adattare il piano escursione.	9
15	Capogruppo	Eliminazione sessione	Come capogruppo voglio poter eliminare una sessione che ho creato, così da annullare l'escursione per tutto il gruppo.	10
33	Amministratore	Gestione registrazioni utenti e ruoli	Come Amministratore, voglio poter gestire le registrazioni degli utenti e assegnare/modificare i loro ruoli, così da controllare i privilegi di accesso al sistema.	13
7	Partecipante	Checklist equipaggiamento dinamica	Come escursionista, voglio ricevere una checklist dell'equipaggiamento necessaria basata sull'itinerario scelto e sulle condizioni meteo (durante i giorni precedenti alla partenza), così da prepararmi adeguatamente.	15
continua...				

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
19	Capogruppo	Generazione codice invito QR	Come Capogruppo, voglio poter generare un codice invito alfanumerico e un QR Code per il gruppo, così da permettere ai partecipanti di unirsi facilmente.	17
9	Partecipante	Unione a escursione di gruppo	Come escursionista, voglio poter unirmi a un'escursione di gruppo inserendo il codice invito fornito dal Capogruppo, così da partecipare all'escursione in modo coordinato.	18
12	Partecipante	Mappa offline e bussola	Come escursionista, voglio poter consultare una mappa offline con la mia posizione e una bussola, così da orientarmi anche senza copertura internet.	20
10	Partecipante	Tracciamento GPS in background	Come escursionista, voglio che la mia posizione GPS venga tracciata automaticamente in background durante l'escursione, così da garantire la mia sicurezza e quella del gruppo.	22
13	Partecipante	Auto-Pause GPS da fermo	Come escursionista, voglio che il tracciamento GPS si riduca automaticamente quando sono fermo, così da risparmiare la batteria del dispositivo.	24
42	Sistema	Foreground Service per tracking continuo	Come sistema, voglio mantenere attivo il tracciamento GPS e l'ascolto BLE tramite Foreground Service, così da evitare che Android interrompa i processi critici (Doze Mode).	26
11	Partecipante	Invio segnale SOS	Come escursionista in difficoltà, voglio poter inviare un segnale SOS con le mie coordinate GPS, così da ricevere soccorso il prima possibile.	28
34	Sistema	Risposta API < 500 ms (p95)	Come sistema, voglio che le query geospaziali vengano elaborate in meno di 500 ms al 95° percentile in condizioni 4G, così da garantire un'esperienza utente fluida.	35
22	Capogruppo	Ricezione e validazione SOS	Come Capogruppo, voglio ricevere i segnali SOS dei partecipanti e poterli validare tramite la mia dashboard, così da attivare i soccorsi solo in caso di reale necessità.	38
36	Sistema	Cifratura AES-256 comunicazioni BLE	Come sistema, voglio che tutti i payload SOS e della chat siano cifrati con AES-256 prima del broadcast BLE, così da garantire la riservatezza delle comunicazioni.	40

continua...

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
39	Sistema	Download preventivo Hike Packet	Come sistema, voglio scaricare preventivamente la traccia GeoJSON e le Map Tiles OSM con padding di 1 km, così da garantire la navigazione offline in zona di not-spot.	42
20	Capogruppo	Dashboard tracking GPS in tempo reale	Come Capogruppo, voglio poter visualizzare la posizione GPS di tutti i partecipanti in tempo reale sulla mappa, così da monitorare la coesione del gruppo.	45
40	Sistema	Validazione TTL e firma ECC segnale SOS	Come sistema, voglio che i segnali SOS nella rete BLE siano accettati solo se hanno firma ECC valida e $\text{hopCount} \leq 10$, così da prevenire loop e spoofing.	48
21	Capogruppo	Gestione emergenze e broadcast allarmi	Come Capogruppo, voglio poter ricevere un allarme in broadcast da tutto il gruppo (online/beacon BLE) così da gestire le emergenze e coordinare la risposta.	52
23	Capogruppo	Inoltro SOS ai soccorsi (CNSAS)	Come Capogruppo, voglio poter inoltrare un segnale SOS validato alle autorità di soccorso (CNSA-S/112), così da attivare i soccorsi professionali in modo sicuro.	56
24	Capogruppo	Failover automatico leadership	Come partecipante, voglio che in caso di indisponibilità del Capogruppo per 10+ minuti, venga eletto automaticamente un nuovo leader, così da garantire la continuità della gestione delle emergenze.	60
14	Partecipante	Visualizzazione posizione relativa al gruppo	Come escursionista, voglio vedere la posizione degli altri membri del gruppo sulla mappa, così da sapere dove si trovano rispetto a me.	64
37	Sistema	Sincronizzazione batch Event Sourcing	Come sistema, voglio sincronizzare gli eventi di gamification accumulati offline tramite Event Sourcing in batch, così da garantire la consistenza del saldo Crediti Sociali.	68
26	Capogruppo	Sincronizzazione batch Event Sourcing	Come escursionista che ha premuto SOS per errore, voglio poter annullare il segnale prima che venga inoltrato, così da evitare l'attivazione inutile dei soccorsi.	72
continua...				

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
38	Sistema	Idempotenza transazioni crediti (UUID v4)	Come sistema, voglio garantire che ogni evento di gamification sia processato una sola volta, così da prevenire il double-spending dei Crediti Sociali.	76
17	Partecipante	Ricezione alert meteo e pericoli	Come escursionista, voglio ricevere notifiche di allerta meteo o pericoli nel percorso, così da prendere decisioni informate sulla mia sicurezza.	80
27	Gestore Rifugio	Invio allerta pericolo push	Come Gestore Rifugio, voglio poter inviare notifiche push di allerta pericolo agli escursionisti nella zona, così da informarli tempestivamente di situazioni pericolose.	84
25	Sistema	Risoluzione conflitto doppia leadership	Come sistema, voglio risolvere automaticamente i conflitti quando due partizioni mesh hanno leader distinti, così da garantire un'unica leadership coerente al ricongiungimento.	88
43	Sistema	Timeout API Meteo max 3 secondi	Come sistema, voglio che il polling alle API di MeteoTrentino abbia un timeout massimo di 3 secondi, così da non bloccare l'utente in caso di lentezza del servizio esterno.	92
28	Gestore Rifugio	Monitoraggio telemetria macchinari IoT	Come Gestore Rifugio, voglio poter monitorare lo stato dei macchinari (compattatori, disidratatori) tramite dashboard, così da gestire la manutenzione preventiva.	96
35	Sistema	Storage locale SQ-Lite max 50 MB con FIFO	Come sistema, voglio che il database locale non superi 50 MB applicando una politica FIFO, così da non saturare lo storage del dispositivo mobile.	100
16	Partecipante	Consultazione dati di navigazione	Come escursionista, voglio poter consultare i dati di navigazione (quota, velocità, distanza percorsa) durante l'escursione, così da monitorare il mio progresso.	104
41	Sistema	Compatibilità Android 9.0+ (API 28)	Come sistema, voglio essere compatibile con dispositivi Android 9.0 o superiori, così da coprire la maggior parte del parco dispositivi degli utenti.	108
continua...				

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
29	Gestore Rifugio	Reset macchinari IoT	Come Gestore Rifugio, voglio poter resettare i macchinari IoT del rifugio tramite l'app, così da risolvere malfunzionamenti senza intervento tecnico in loco.	112
32	Amministratore	Monitoraggio spesa crediti sociali	Come Amministratore, voglio poter monitorare il volume di Crediti Sociali emessi e spesi sulla piattaforma, così da verificare la sostenibilità del sistema economico.	125
30	Gestore Rifugio	Monitoraggio affollamento rifugio	Come Gestore Rifugio, voglio poter visualizzare il conteggio delle persone presenti nel rifugio tramite sensori ottici, così da gestire la capacità in modo efficiente.	130
31	Amministratore	Gestione promozioni partner	Come Amministratore, voglio poter inserire e gestire le promozioni dei partner commerciali, così da offrire sconti agli utenti che spendono i loro Crediti Sociali.	135
8	Partecipante	Reindirizzamento store partner	Come escursionista, voglio poter essere reindirizzato agli store partner per acquistare l'attrezzatura mancante, così da completare il mio equipaggiamento comodamente.	140
45	Partecipante	Visualizzazione difficoltà e dislivello	Come escursionista, voglio vedere la difficoltà tecnica e il dislivello di una attività, così da valutare se è adatto alle mie capacità in modo chiaro sull'interfaccia grafica in app.	150
44	Tutti gli utenti	Recupero password	Come utente che ha dimenticato la password, voglio poter richiedere il reset tramite email, così da riacquisire accesso al mio account.	160
46	Partecipante	Abbandono sessione escursione	Come escursionista, voglio poter abbandonare una sessione di gruppo quando lo desidero, così da gestire autonomamente la mia partecipazione.	170

Tabella 3: Product Backlog completo con priorità (Importanza).

2.3 Definizione di “Done”

Una User Story è dichiarata **Done** se soddisfa *tutti* i seguenti criteri:

1. **Completamento funzionale:** Il comportamento descritto è implementato ed è verificabile sul ramo di integrazione (UI).
2. **Qualità del codice:** Il codice è completo di commenti dove necessario (logica non ovvia, scelte architetturali, edge case) e segue gli standard concordati dal team.
3. **Integrazione nel repository:** Il lavoro è mergiato nel branch condiviso, senza conflitti irrisolti.
4. **Build e stabilità:** Il progetto Android compila senza errori (`./gradlew assembleDebug`); il backend parte senza eccezioni (`npm run dev`).
5. **Verifica manuale:** È stata eseguita una prova del flusso principale della story (da chi ha sviluppato o tramite peer-review). Bug evidenti al primo utilizzo invalidano lo stato di Done.
6. **Tracciabilità:** La story è aggiornata nello strumento di sprint (Product/Sprint Backlog `xlsx`).
7. **Documentazione leggera:** Se la story introduce setup nuovo o variabili d’ambiente, è aggiornato almeno un documento del team (`docs/setup_mobile.md` o `TSM_PROJECT_STATE.md`).
8. **API contract:** Ogni nuovo endpoint è descritto in Swagger (`swagger-output.json`); ogni endpoint che tocca dati utente è protetto dal middleware `authenticate`.

3 Sezione Sprint #1

3.1 Goal

“Consegnare un’app TSM dimostrabile end-to-end che permetta a un capogruppo di pianificare un’escursione da GPX, condividere un codice invito, accogliere partecipanti, avviare il tracking GPS, completare l’attività e ritrovarla in ‘Le mie attività’, con un backend Node.js + MongoDB autenticato via JWT e integrato con un servizio meteo reale (TINIA).”

3.2 Sprint Planning (Sprint Backlog)

Parametri dello Sprint

Parametro	Valore
Durata	settimana (10/05/2026 – 17/05/2026)
Capacità team	3 membri $\times$ $\sim$ 10h/settimana $\approx$ 210h totali
Story points pianificati	55
Story points completati	$\sim$ 50

Tabella 4: Parametri generali dello Sprint 1.

Andamento dello sprint

Settimana	SP Pianificati	SP Completati	Note principali
SS1	13	12	Auth completo (register/login JWT/verify email); registrazione rifugio in progress
SS2	16	17	SessionHub PIANIFICA + UNISCITI; integrazione meteo TINIA; generazione QR
SS3	14	14	SessionDetail + edit mode + profilo altimetrico Canvas + GPS tracking engine
SS4	12	7	ActivityList + Room + bugfix Sprint; 5 SP in debito (SOS backend, refinements)

Tabella 5: Distribuzione degli story point pianificati e completati nella settimana dello Sprint 1.

$SS_n \rightarrow$ sub-sprint numero.

Sprint Backlog

					Estimated Effort Remaining						
ID	Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7
3	Come utente voglio poter creare un account inserendo i miei dati, così da accedere alla piattaforma usufruire dei suoi servizi.	1. Sviluppo UI registrazione utente con validazione input.	Federico	2	2	1	-	-	-	-	-
3		2. Setup endpoint API per registrazione e hashing password.	Marco	2	2	2	2	2	-	-	-
3		3. Implementazione invio email di verifica.	Giacomo	3	3	2	1	-	-	-	-
3		4. Integrazione DB per persistenza dati utente.	Marco	2	2	2	2	-	-	-	-
3		5. Progettazione e implementazione collezione utente nel DataBase	Marco	3	3	3	3	-	-	-	-
3		6. Deep link e flusso verifica email (tsm://auth).	Giacomo	2	2	2	2	2	-	-	-
3		7. Registrazione rifugio (UI campi CAI e coordinate).	Giacomo	2	2	1	-	-	-	-	-
3		8. Documentazione comunicazione Android-server.	Federico	1	1	1	1	1	-	-	-
3		9. Testing & debugging	Marco	1	1	2	2	2	1	-	-
3		10. Documentazione API	Marco	1	1	1	1	-	-	-	-
3		11. Deploy	Federico	1	1	1	1	1	1	-	-
1	Come utente voglio poter effettuare login con le mie credenziali così da accedere alle funzionalità del sistema in al mio account	1. Progettazione e implementazione collezione utente nel DataBase (MongoDB).	Marco	3	3	2	-	-	-	-	-
1		2. Sviluppo UI form di login.	Federico	2	2	2	-	-	-	-	-
1		3. Integrazione login/registrazione con API backend (Retrofit).	Federico	2	2	1	-	-	-	-	-
1		5. Implementazione gestione token JWT in Local Storage/Keychain.	Federico	1	1	1	1	1	-	-	-
1		6. Implementazione gestione Token Backend server.	Marco	2	2	2	-	-	-	-	-
1		7. AuthInterceptor Bearer JWT su chiamate protette.	Federico	1	1	1	1	-	-	-	-
1		8. Reset password via email (forgot/reset flow).	Giacomo	2	2	2	2	1	-	-	-
1		9. Testing del flusso di login.	Giacomo	1	1	-	-	-	-	-	-
1		10. Documentazione	Giacomo	1	1	1	1	1	-	-	-
1		11. Deploy	Giacomo	1	1	1	1	1	-	-	-
4	Come utente autenticato voglio poter effettuare il logout dall'app, così da proteggere i miei dati su dispositivi condivisi.	1. UI pulsante Logout in ProfileScreen e RefugeMainScreen.	Federico	1	1	1	1	1	1	-	-
4		2. Pulizia TokenStorage, cache Room profilo e navigazione verso auth.	Federico	2	2	2	2	2	-	-	-
4		4. Testing flusso logout.	Giacomo	1	1	-	-	-	-	-	-
4		5. Documentazione	Federico	1	1	-	-	-	-	-	-
4		6. Deploy	Federico	1	1	-	-	-	-	-	-

continua...

ID	Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7
2	Come utente già autenticato, voglio poter accedere all'app anche senza connessione internet, così da usare le funzionalità anche in montagna.	1. Caching stato autenticazione e token validi in EncryptedSharedPreferences.	Federico	3	3	3	3	-	-	-	-
2		2. Ripristino sessione JWT all'avvio app (AuthSession).	Federico	2	2	2	2	2	-	-	-
2		3. Interceptor HTTP e gestione errori offline.	Federico	2	2	2	2	2	1	-	-
2		4. Cache profilo utente con Room.	Federico	2	2	2	2	2	2	1	-
2		5. Gestione state-machine dell'app offline/online.	-	2	-	-	-	-	-	-	-
2		6. Testing	Federico	1	1	1	1	1	1	1	-
2		7. Documentazione	Federico	1	1	1	1	1	1	1	-
2		8. Deploy	Federico	1	1	1	1	1	1	1	-
18	Come Capogruppo, voglio poter creare nuova sessione di escursione, parametri del percorso e coordinare il gruppo avviare tracciamento	1. Progettazione schema DB (Collezione Sessione, Route, Partecipante).	Giacomo	3	3	2	-	-	-	-	-
18		2. Sviluppo endpoint REST per inizializzazione sessione.	Giacomo	2	2	2	-	-	-	-	-
18		3. Logica di associazione Capogruppo-Sessione.	Marco	2	2	2	2	-	-	-	-
18		4. Vincolo una sola sessione ACTIVE per utente.	Marco	1	1	1	1	1	-	-	-
18		5. Import e parsing file GPX (stats e profilo altimetrico).	Giacomo	3	3	3	3	2	-	-	-
18		6. UI form PIANIFICA (data, ora, partecipanti, anteprima codice).	Giacomo	3	3	3	3	3	2	-	-
18		7. Flusso AVVIA escursione e PATCH status ACTIVE.	Giacomo	2	2	2	2	2	2	-	-
18		8. Coordinamento AVVIA -> tab Registra (SessionStartCoordinator).	Federico	1	1	1	1	1	1	-	-
18		9. Testing API	Marco	1	1	-	-	-	-	-	-
18		10. Documentazione	Giacomo	1	1	-	-	-	-	-	-
18		11. Deploy	Federico	1	1	1	1	1	1	-	-
6	Come capogruppo voglio poter aggiornare i parametri della sessione (nome, data, ora, partecipanti), così da adattare il piano escursione.	1. Endpoint PATCH /api/v1/sessions/:id.	Marco	3	3	3	2	-	-	-	-
6		2. UI edit mode in SessionDetailScreen (solo creator).	Giacomo	3	3	3	3	3	2	-	-
6		3. SessionDetailViewModel saveEdit con reload.	Giacomo	2	2	2	2	2	1	1	-
6		4. Testing permessi e validazione campi.	Marco	1	1	1	1	1	1	1	-
6		5. Documentazione	Giacomo	1	1	1	1	1	1	1	-
6		6. Deploy	Federico	1	1	1	1	1	1	1	-
15	Come capogruppo voglio poter eliminare una sessione che ho creato, così da annullare l'escursione per tutto il gruppo.	1. Endpoint DELETE /sessions/:id (solo creator).	Marco	2	2	2	2	-	-	-	-
15		2. UI dialog Elimina vs Abbandona (RemovalMode).	Giacomo	2	2	2	2	2	-	-	-

continua...

ID	Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7
15		3. Propagazione rimozione su lista partecipanti.	Giacomo	1	1	1	1	1	1	–	–
15		4. Testing autorizzazioni 403 per non-creator.	Marco	1	1	1	1	1	1	–	–
15		5. Documentazione	Giacomo	1	1	1	1	1	1	–	–
15		6. Deploy	Giacomo	1	1	1	1	1	1	–	–
33	Come Amministratore, voglio poter gestire le registrazioni degli utenti (assegnare/modificare ruoli), così da controllare i privilegi di accesso al sistema	1. Implementazione middleware RBAC (Role-Based Access Control).	Marco	4	4	4	3	–	–	–	–
33		2. Modelli ruoli (rifugio, groupLeader, admin).	Giacomo	2	2	2	2	2	–	–	–
33		3. Endpoint CRUD utenti con requireRoles admin.	Marco	3	3	3	3	2	–	–	–
33		5. Testing	Marco	1	1	–	–	–	–	–	–
33		6. Documentazione	Marco	1	1	1	1	1	1	1	–
33		7. Deploy	Marco	1	1	1	1	1	1	1	–
7	Come escursionista voglio ricevere una checklist dell'equipaggiamento necessaria basata sull'itinerario e sulle condizioni meteo (durante i giorni precedenti alla partenza), così da prepararmi adeguatamente	1. Integrazione API meteo TINI/meteo.report con persistenza MongoDB.	Marco	4	4	2	–	–	–	–	–
7		2. Modello Location GeoJSON e endpoint forecast 3h/24h.	Marco	3	3	3	3	–	–	–	–
7		3. Integrazione meteo reale in SessionDetailScreen.	Giacomo	2	2	2	2	2	1	–	–
7		4. Sviluppo UI checklist con drag-and-drop e spunta.	Giacomo	2	2	2	2	2	2	1	–
7		5. Sviluppo algoritmo di mappatura meteo/altitudine -> equipaggiamento.	–	4	–	–	–	–	–	–	–
7		6. Testing API	Marco	1	1	1	1	1	–	–	–
7		7. Documentazione	–	1	–	–	–	–	–	–	–
7		8. Deploy	Marco	1	1	1	1	1	1	–	–
19	Come Capogruppo, voglio poter generare un codice invito alfanumerico e un QR Code per gruppo, così da permettere ai partecipanti di unirsi facilmente	1. Endpoint generazione hash invito univoco (TSM-XXXX).	Giacomo	2	2	2	–	–	–	–	–
19		2. Integrazione libreria generazione QR Code a schermo (ZXing).	Giacomo	2	2	2	2	–	–	–	–
19		3. UI InviteCodeCard con copia negli appunti.	Giacomo	1	1	1	1	1	–	–	–
19		4. Sviluppo logica per lettura QR tramite fotocamera.	–	4	–	–	–	–	–	–	–
19		5. Testing	Giacomo	1	1	–	–	–	–	–	–
19		6. Documentazione	Giacomo	1	–	–	–	–	–	–	–
19		7. Deploy	Federico	1	–	–	–	–	–	–	–

continua...

ID	Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7
9	Come escursionista, voglio poter unirmi a un'escursione di gruppo inserendo il codice invito fornito dal Capogruppo, così da partecipare all'escursione in modo coordinato	1. UI input codice invito (OTP boxes tab UNISCITI).	Giacomo	2	2	2	2	2	-	-	-
9		2. Validazione codice lato server e join tabella Partecipanti.	Marco	2	2	2	2	2	-	-	-
9		3. Lista sessioni utente con leave/delete (creator vs partecipante).	Giacomo	2	2	2	2	2	2	-	-
9		4. Download configurazione e mappa sessione in dettaglio.	Federico	2	2	2	2	2	2	1	-
9		5. Testing	Giacomo	1	1	1	1	1	1	-	-
9		6. Documentazione	Giacomo	1	1	1	1	1	1	1	-
9		7. Deploy	Federico	1	-	-	-	-	-	-	-
12	Come escursionista, voglio poter consultare una mappa offline con la mia posizione e una bussola così da orientarmi anche senza copertura internet	1. Integrazione OSMDroid per rendering mappa e marker utente.	Federico	5	5	5	5	5	3	-	-
12		2. UserLocationTracker e indicatore segnale GPS.	Federico	2	2	2	2	2	2	-	-
12		3. Centratura mappa e lifecycle TsmMapView.	Federico	2	2	2	2	2	2	1	-
12		4. Lettura SensorManager (Magnetometro/Accelerometro) per bussola.	Federico	4	4	4	4	4	4	-	-
12		5. Binding sensori-UI per rotazione mappa.	-	3	-	-	-	-	-	-	-
12		6. Download e cache Map Tiles OSM offline.	-	4	-	-	-	-	-	-	-
12		7. Testing	Federico	1	1	1	1	1	1	1	-
12		8. Documentazione	Federico	1	1	1	1	1	1	1	-
12		9. Deploy	Federico	1	1	1	1	1	1	1	-
10	voglio che la mia posizione GPS venga tracciata automaticamente in background durante l'escursione, così da garantire la mia sicurezza e quella del gruppo	1. Integrazione FusedLocationProviderClient.	Federico	4	4	4	4	4	2	-	-
10		2. HikeTrackingEngine e visualizzazione traccia su mappa.	Federico	3	3	3	3	3	3	-	-
10		3. Sviluppo logica di caching locale offline (Room/SQLite telemetria).	Federico	3	3	3	3	3	3	2	-
10		4. Worker periodico per invio batch coordinate al server.	-	3	-	-	-	-	-	-	-
10		5. Testing	Federico	1	1	1	1	1	1	1	-
10		6. Documentazione	-	1	-	-	-	-	-	-	-
10		7. Deploy	-	1	-	-	-	-	-	-	-
13	escursionista, voglio che il tracciamento GPS si riduca automaticamente quando sono fermo, così da risparmiare la batteria del dispositivo	1. StationaryDetector e logica auto-pause.	Federico	3	3	3	3	3	3	-	-
13		2. UI notifiche e metriche durante auto-pausa.	Federico	2	2	2	2	2	2	2	-

continua...

ID	Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7
13		3. Integrazione Activity Recognition API.	Federico	3	3	3	3	3	3	2	–
13		4. Profilazione consumo batteria (Android Profiler).	–	2	–	–	–	–	–	–	–
13		5. Testing	Federico	1	1	1	1	1	1	1	–
13		6. Documentazione	–	1	–	–	–	–	–	–	–
13		7. Deploy	–	1	–	–	–	–	–	–	–
42	Come sistema voglio mantenere attivo tracciamento GPS tramite Foreground Services così da evitare che Android interrompa i processi (Doze Mode)	1. Implementazione Android Foreground Service con notifica persistente.	Federico	4	4	4	4	4	4	2	–
42		2. Richiesta permessi a runtime (Location).	Federico	2	2	2	2	2	2	–	–
42		3. Configurazione WakeLock e ottimizzazione Doze Mode.	Federico	3	3	3	3	3	3	2	–
42		4. Test del ciclo di vita del servizio in background.	–	3	–	–	–	–	–	–	–
42		5. Documentazione	–	1	–	–	–	–	–	–	–
42		6. Deploy	–	1	–	–	–	–	–	–	–
11	Come escursionista in difficoltà voglio poter inviare un segnale SOS con le mie coordinate GPS così da ricevere soccorso il prima possibile.	1. UI pulsante SOS (dialog conferma in RegistraScreen).	Giacomo	2	2	2	2	2	2	1	–
11		2. Sviluppo logica di invio payload BLE e HTTP/MQTT.	–	3	–	–	–	–	–	–	–
11		3. Endpoint POST /api/v1/emergencies backend.	–	2	–	–	–	–	–	–	–
11		4. Gestione accodamento invio in assenza di rete.	–	1	–	–	–	–	–	–	–
11		5. Testing	–	1	–	–	–	–	–	–	–
11		6. Documentazione	–	1	–	–	–	–	–	–	–
11		7. Deploy	–	1	–	–	–	–	–	–	–
34	Come sistema voglio che le query geospaziali vengano elaborate in meno di 500 ms al 95 percentile (condizioni 4G), così da garantire un'esperienza utente fluida	1. Creazione indici spaziali MongoDB (sessions/locations).	Marco	3	3	2	–	–	–	–	–
34		2. Ottimizzazione query di prossimità.	–	4	–	–	–	–	–	–	–
34		3. Load testing e profiling DB.	–	3	–	–	–	–	–	–	–
34		4. Testing	–	1	–	–	–	–	–	–	–
34		5. Documentazione	–	1	–	–	–	–	–	–	–
34		6. Deploy	–	1	–	–	–	–	–	–	–
–	Setup progetto e infrastruttura condivisa (monorepo, CI, documentazione)	1. Setup monorepo, GitFlow e documentazione iniziale.	Giacomo	3	3	2	–	–	–	–	–
–		2. Docker Compose MongoDB e Mosquitto.	Giacomo	2	2	2	–	–	–	–	–
–		3. Boilerplate backend Express e struttura mobile MVVM.	Giacomo	2	2	2	2	–	–	–	–
–		4. Bottom navigation app (Sessione, Mappa, Profilo).	Federico	2	2	2	2	2	–	–	–

continua...

ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7
–	5. Setup Swagger e documentazione endpoint API.	Marco	2	1	–	–	–	–	–	–
–	6. Ristrutturazione repository backend (app.js, middleware).	Marco	3	1	–	–	–	–	–	–
TOTALE			252	196	176	150	119	79	30	–

Tabella 6: Sprint Backlog completo con stime iniziali ed effort rimanente per giorno.

Lo Sprint Backlog completo è disponibile in "docs/SCRUM_Logs_Excel /Backlog V1 - Sprint 1 Backlog.xlsx" nel repository.

Burndown Chart

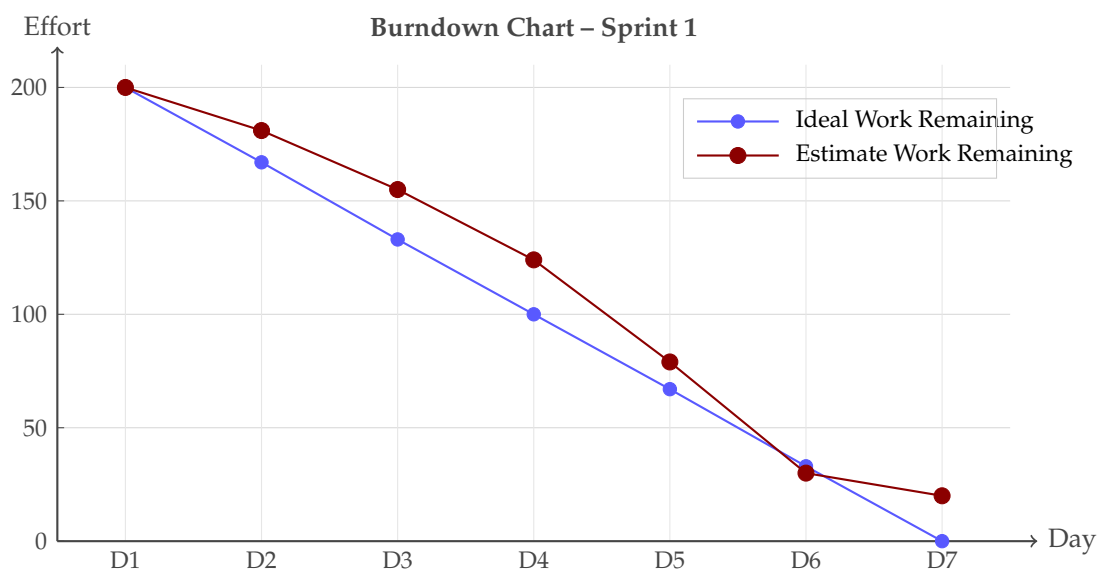


Figura 1: Burndown Chart Sprint 1. L'effort iniziale è normalizzato a 200 unità. Il team ha mantenuto un ritmo leggermente superiore all'ideale nei primi 4–5 giorni, completando circa 50/55 story points pianificati.

3.3 Test Cases

Per Sprint 1 è sufficiente il solo **design** dei test case. La tabella seguente copre i flussi principali e documenta esplicitamente anche i bug critici identificati durante l'audit di fine sprint: **TC-36** (Bug C1 – partecipante non può avviare) e **TC-47** (Bug C3 – tracking GPS background interrotto). Il Bug C2 (endpoint Weather non protetti) è stato identificato tramite revisione del codice: TC dedicati saranno aggiunti in Sprint 2 (US-17).

TC	US	Tipo	Passi / Input	Output atteso
TC-01	US-01	API (Postman)	POST /users body: username, email, pwd, role=groupLeader; account non esistente	HTTP 201; utente creato con isVerified:false; email di verifica inviata.
TC-02	US-01	API (Postman)	POST /users body: email già registrata	HTTP 409 Collisione rilevata: Email o Username già utilizzati.
TC-03	US-01	API (Postman)	POST /users body: senza campo role	HTTP 201; utente salvato con role=groupLeader (default schema).
TC-04	US-01	API (Postman)	POST /users body: username già registrato, email diversa	HTTP 409 Collisione rilevata: Email o Username già utilizzati.
TC-05	US-01	API (Postman)	POST /users body: username vuoto/assente	HTTP 500; Mongoose Path 'username' is required. Nessun utente creato.
TC-06	US-01	API (Postman)	POST /users body: email vuota/assente	HTTP 500; Mongoose Path 'email' is required. Nessun utente creato.
TC-07	US-01	API (Postman)	POST /users body: password vuota/assente	HTTP 500; bcrypt.hash(undefined) lancia eccezione. Nessun utente creato.
TC-08	US-01	API (Postman)	POST /users body: role=superuser (fuori enum)	HTTP 500; errore enum Mon- goose (groupLeader, rifugio, admin).
TC-09	US-01	API (Postman)	POST /users body valido; trasporto SMTP non disponibile	HTTP 201; utente persistito (isVerified:false); errore SMTP loggato lato server, non propagato al client.
TC-10	US-02	API (Postman)	POST /auth/login body: email verificata, password corretta	HTTP 200; JWT token restituito nel body.
TC-11	US-02	API (Postman)	POST /auth/login body: email non registrata	HTTP 401 Invalid email.
TC-12	US-02	API (Postman)	POST /auth/login body: email corretta, password errata	HTTP 401 password is invalid.
TC-13	US-02	API (Postman)	POST /auth/login body: credenziali corrette, isVerified:false	HTTP 403 Accesso negato. Eseguire la verifica SMTP inviata via email.
TC-14	US-02	API (Postman)	POST /auth/login body: email="", password valida	HTTP 401 Invalid email (findOne su stringa vuota restituisce null).
continua...				

TC	US	Tipo	Passi / Input	Output atteso
TC-15	US-02	API (Postman)	POST /auth/login body: email verificata, password=""	HTTP 401 password is invalid (bcrypt.compare ritorna false).
TC-16	US-16	UI manuale	JWT valido in local storage 1) Tap pulsante Logout	JWT rimosso da local storage; redirect a schermata login; nessuna chiamata autenticata eseguibile.
TC-17	US-16	UI manuale	Nessun JWT in local storage 1) Naviga all'azione Logout	Redirect a schermata login; nessun errore; nessuna chiamata API effettuata.
TC-18	US-10	Device fisico	Debug locale Android Studio (Foreground Service GPS)	Da documentare in Sprint 2.
TC-19	US-05	API (Postman)	POST /api/v1/sessions header: Authorization: Bearer <JWT> body: routeDetails.name, difficultyLevel, data, ora, posti	HTTP 201; sessione PLANNED con inviteCode TSM-XXXX; creator aggiunto ai partecipanti con role:groupLeader.
TC-20	US-05	API (Postman)	POST /api/v1/sessions body senza routeDetails.name	HTTP 400 routeDetails.name obbligatorio. Nessuna sessione creata.
TC-21	US-05	API (Postman)	POST /api/v1/sessions utente già in sessione ACTIVE	HTTP 409 Hai una sessione attualmente in corso (tracciamento ATTIVO). Concludila prima di crearne un'altra.
TC-22	US-05	API (Postman)	POST /api/v1/sessions body: difficultyLevel=EXTREME (fuori enum)	HTTP 500; errore enum Mongoose (T, E, EE, EEA). Nessuna sessione creata.
TC-23	US-05	API (Postman)	POST /api/v1/sessions senza header Authorization	HTTP 401; middleware authenticate rifiuta. Nessuna sessione creata.
TC-24	US-06	API (Postman)	POST /api/v1/sessions/join header: Authorization: Bearer <JWT> body: inviteCode: TSM-7A4F; sessione PLANNED, utente non partecipante	HTTP 200; utente aggiunto con role:hiker; sessionRoles aggiornato.
TC-25	US-06	API (Postman)	POST /api/v1/sessions/join body: inviteCode: TSM-0000 (inesistente)	HTTP 404 Codice invito non valido.
TC-26	US-06	API (Postman)	POST /api/v1/sessions/join body: codice valido; sessione in stato ACTIVE	HTTP 409 La sessione non è più aperta.
TC-27	US-06	API (Postman)	POST /api/v1/sessions/join utente già partecipante alla sessione	HTTP 409 Sei già in questa sessione.
TC-28	US-06	API (Postman)	POST /api/v1/sessions/join body vuoto {}	HTTP 400 inviteCode mancante nel body.

continua...

TC	US	Tipo	Passi / Input	Output atteso
TC-29	US-06	API (Postman)	POST /api/v1/sessions/join utente in sessione ACTIVE; target sessione PLANNED	HTTP 409 Hai una sessione attualmente in corso (tracciamento ATTIVO). Concludila prima di unirti a una nuova.
TC-30	US-20	API (Postman)	GET /api/v1/sessions/my header: Authorization: Bearer <JWT>	HTTP 200; array sessioni (creator o partecipante) ordinato per meetingDate; [] se nessuna.
TC-31	US-20	API (Postman)	GET /api/v1/sessions/<validId> header: Authorization: Bearer <JWT>	HTTP 200; sessione completa con creatorId e participants.userId popolati.
TC-32	US-20	API (Postman)	GET /api/v1/sessions/ (ObjectId valido, nessun documento)	HTTP 404 Sessione non trovata.
TC-33	US-20	API (Postman)	GET /api/v1/sessions/not-a-valid-id (stringa non castabile a ObjectId)	HTTP 400 ID non valido; CastError intercettato.
TC-34	US-09	API (Postman)	PATCH /api/v1/sessions/<id>/status header: Authorization: Bearer <creatorJWT> body: {status: ACTIVE}; sessione in PLANNED	HTTP 200; status=ACTIVE; startTime impostato al timestamp corrente.
TC-35	US-09	API (Postman)	PATCH /api/v1/sessions/<id>/status header: Authorization: Bearer <creatorJWT> body: {status: COMPLETED}; sessione in ACTIVE	HTTP 200; status=COMPLETED; endTime impostato al timestamp corrente.
TC-36	US-08	API (Postman)	PATCH /api/v1/sessions/<id>/status header: Authorization: Bearer <hikerJWT> (non creator) body: {status: ACTIVE}	Atteso: HTTP 403 Solo il Capogruppo può modificare la sessione. Lato mobile – Bug C1: 403 silente, caricamento perpetuo.
TC-37	US-08	API (Postman)	PATCH /api/v1/sessions/<id>/status body: {status: PAUSED} (fuori enum)	HTTP 400 Status non valido. Valori accettati: PLANNED, ACTIVE, COMPLETED, CANCELLED.
TC-38	US-08	API (Postman)	PATCH /api/v1/sessions/<id>/status body vuoto {}	HTTP 400 Status non valido. Valori accettati: PLANNED, ACTIVE, COMPLETED, CANCELLED.
TC-39	US-08	API (Postman)	PATCH /api/v1/sessions/<id> header: Authorization: Bearer <creatorJWT> body: nome, data, ora, location, posti, livello	HTTP 200; campi aggiornati (patch semantics); inviteCode invariato.
continua...				

TC	US	Tipo	Passi / Input	Output atteso
TC-40	US-08	API (Postman)	PATCH /api/v1/sessions/<id> header: Authorization: Bearer <hikerJWT> (non creator) body: {meetingDate: 2026-08-01}	HTTP 403 Solo il Capogruppo può modificare la sessione.
TC-41	US-19	API (Postman)	POST /api/v1/sessions/<id>/leave header: Authorization: Bearer <hikerJWT> utente partecipante (non creator)	HTTP 200; utente rimosso dai partecipanti; sessione aggiornata restituita.
TC-42	US-19	API (Postman)	POST /api/v1/sessions/<id>/leave header: Authorization: Bearer <creatorJWT>	HTTP 403 Il Capogruppo non può abbandonare la sessione. Eliminala se vuoi rimuoverla.
TC-43	US-17	API (Postman)	DELETE /api/v1/sessions/<id> header: Authorization: Bearer <creatorJWT>	HTTP 200 Sessione eliminata; documento rimosso da MongoDB.
TC-44	US-17	API (Postman)	DELETE /api/v1/sessions/<id> header: Authorization: Bearer <hikerJWT> (non creator)	HTTP 403 Solo il Capogruppo può eliminare la sessione.
TC-45	US-10	Device fisico	Debug locale Android Studio (Auto-pause GPS)	Da documentare in Sprint 2.
TC-46	US-07	Device fisico	Debug locale Android Studio (Checklist equipaggiamento meteo)	Da documentare in Sprint 2.
TC-47	US-10	Device fisico	Tracking attivo 1) Tap AVVIA 2) Blocca schermo per 5 min 3) Sblocca schermo	Atteso: distanza/quota aggiornate correttamente. Attuale – Bug C3: traccia GPS troncato (permesso ACCESS_BACKGROUND_LOCATION mancante).
TC-48	US-18	API (Postman)	PUT /users/<id> header: Authorization: Bearer <adminJWT> body: {role: rifugio}	HTTP 200; role aggiornato; passwordHash e __v esclusi dalla risposta.
TC-49	US-18	API (Postman)	PUT /users/<id> header: Authorization: Bearer <adminJWT> body: {role: superadmin} (fuori enum)	HTTP 500; errore enum Mon-goose (groupLeader, rifugio, admin).
TC-50	US-18	API (Postman)	PUT /users/<id> header: Authorization: Bearer <groupLeaderJWT> body: {role: admin}	HTTP 403 Forbidden; requireRoles("admin") rifiuta prima di updateUser.

continua...

TC	US	Tipo	Passi / Input	Output atteso
TC-51	US-18	API (Postman)	PUT /users/<id> senza header Authorization; body: {role: rifugio}	HTTP 401 No token provided; middleware authenticate rifiuta.
TC-52	US-18	API (Postman)	PUT /users/ header: Authorization: Bearer <adminJWT> (ObjectId valido, nessun documento)	HTTP 404 User not found.
TC-53	US-18	API (Postman)	PUT /users/not-a-valid-id header: Authorization: Bearer <adminJWT>	HTTP 400 Invalid user ID format; CastError intercettato.
TC-54	US-18	API (Postman)	PUT /users/<id> body: username già usato da altro utente	HTTP 409 Username or email already in use; utente non modificato.
TC-55	US-18	API (Postman)	PUT /users/<id> body vuoto {}	HTTP 200; utente restituito invariato (loop allowedUpdates non applica modifiche).
TC-56	US-18	API (Postman)	DELETE /users/<id> header: Authorization: Bearer <adminJWT>	HTTP 200 User deleted successfully; documento rimosso da MongoDB.
TC-57	US-18	API (Postman)	DELETE /users/<id> header: Authorization: Bearer <groupLeaderJWT>	HTTP 403 Forbidden; requireRoles("admin") rifiuta.
TC-58	US-18	API (Postman)	DELETE /users/ header: Authorization: Bearer <adminJWT>	HTTP 404 User not found.
TC-59	US-18	API (Postman)	DELETE /users/not-a-valid-id header: Authorization: Bearer <adminJWT>	HTTP 400 Invalid user ID format; CastError intercettato.
TC-60	US-05	API (Postman)	POST /api/v1/sessions header: Authorization: Bearer <JWT> body: routeDetails.name valido	HTTP 201; inviteCode corrisponde a ^TSM-[0-9A-F]{4}\$ (es. TSM-7A4F).
TC-61	US-05	API (Postman)	Due POST /api/v1/sessions indipendenti (utenti A e B, nessuno in sessione ACTIVE)	Entrambe HTTP 201; i due inviteCode sono valori distinti.
TC-62	US-05	API (Postman)	PATCH /api/v1/sessions/<id> header: Authorization: Bearer <creatorJWT> body: {inviteCode: TSM-0000, meetingLocation: Piazzale Roma}	HTTP 200; meetingLocation aggiornata; inviteCode invariato (immutabile by design).

Tabella 7: Test case di Sprint 1 (design). I TC-18, TC-45, TC-46 e TC-47 richiedono completamento in Sprint 2.

La suite completa dei test case (incluse precondizioni, esiti effettivi e cronologia bug fixati) è disponibile nel file docs/SCRUM_Logs_Excel/Backlog V1 - Test cases.xls e nel documento docs/SCRUM_Logs_Excel/test_cases_sprint1.md nel repository.

3.4 Sprint Review

La Sprint Review si è svolta alla fine della settimana con una demo live di circa 15 minuti che ha coperto il seguente flusso completo:

1. **(0:00–1:30) Autenticazione completa:** Registrazione nuovo utente → ricezione mail SMTP → tap su deep link `tsm://auth/verify/...` → login con JWT → accesso all'area principale con bottom navigation.
2. **(1:30–5:30) Pianificazione escursione:** Import file GPX reale (es. *Catinaccio.gpx*) → profilo altimetrico generato automaticamente (Canvas con area fill gradient) → stima punti CAI → generazione codice TSM-XXXX.
3. **(5:30–8:00) Join partecipante (secondo account):** Login su secondo dispositivo → tab UNISCITI → inserimento codice → apertura `SessionDetailScreen`: meteo reale TINIA (forecast 3h/24h), checklist drag-and-drop, lista partecipanti con avatar.
4. **(8:00–9:00) Avvio e tracking GPS:** Capogruppo: tap “AVVIA” → switch automatico a tab Registra → mappa OSMdroid con traccia GPS in tempo reale → metriche (distanza, dislivello, quota, tempo) → Stop → status COMPLETED persistito su MongoDB.
5. **(9:00–12:30) Le mie attività:** Home → tab “Le mie attività” → card statistiche annuali → dettaglio attività con metriche e difficoltà CAI calcolata con formula Naismith + modello TSM.
6. **(12:30–15:00) Swagger API Docs:** Apertura di `http://localhost:3000/api-docs` → esplorazione degli endpoint implementati (auth, users, sessions, weather) con schemi request/response.

Aspetti rilevanti emersi dalla discussione post-demo:

- Il profilo altimetrico reale dal GPX è stato il feature più apprezzato: differenzia concretamente TSM da Komoot/AllTrails su sessioni proprie del team.
- Il modello di stima CAI (`HikeEstimation.kt`: formula polinomiale su pendenza + equivalenza Naismith) è stato riconosciuto come valore differenziale rispetto alle app concorrenti.
- Sono stati identificati 3 bug critici (Bug C1, C2, C3) durante l'audit interno di fine sprint, **tutti corretti il 17/05** prima della chiusura. I dettagli implementativi dei fix sono documentati nella Sprint Retrospective (§3.6).
- Il sistema meteo TINIA – integrazione con geospatial 2dsphere e cache MongoDB 1h – ha funzionato correttamente su dati reali del Trentino.

3.5 Product Backlog Refinement

A seguito della Sprint Review, il team ha aggiornato il Product Backlog con le seguenti nuove User Story per Sprint 2, derivate dai bug critici e dal debito tecnico identificato nell'audit interno:

Priorità	User Story / Motivazione
Must	Fix Bug C1 (✓ applicato 17/05) – Pulsante AVVIA ora visibile solo al creator (isCreator check in SessionDetailScreen); i partecipanti vedono un chip informativo “In attesa del Capogruppo”.
Must	Fix Bug C2 (✓ applicato 17/05) – POST /seed e POST /refresh ora richiedono authenticate + requireRoles('admin'); endpoint debug GET /test rimosso.
Must	Fix Bug C3 (✓ applicato 17/05) – ACCESS_BACKGROUND_LOCATION e WAKE_LOCK aggiunti all'AndroidManifest. Flusso runtime Android 10+ da implementare in Sprint 2.
Should	Backend SOS – Implementare POST /api/v1/emergencies con firma ECC per chiudere RF9 (oggi solo UI).
Could	HomeScreen Social Feed – Feed community e attività sociali (schermata oggi placeholder).
Should	WorkManager store-and-forward – Upload batch telemetria GPS offline al ripristino rete.
Could	Socket.io real-time – Posizioni live del gruppo nella dashboard capogruppo (RF12).
Could	BLE Mesh fallback SOS – Propagazione SOS offline via BLE Mesh (RF13).

Tabella 8: Product Backlog Refinement: nuove user story emerse durante lo Sprint 1 e priorità per lo Sprint 2.

Variazioni al Product Backlog esistente:

- I task relativi alla raccolta fisica dei rifiuti (RF15–RF18 di D1) erano già stati soppressi in D2 e rimangono fuori scope.
- La generazione di coppie di chiavi ECC lato Android Keystore (originariamente nella bozza di Sprint 1) viene posticipata a Sprint 3 per complessità crittografica superiore alle stime iniziali.
- US-07 viene ri-prioritizzata (Should → Must) per Sprint 2 in funzione del completamento dell'algoritmo meteo/equipaggiamento automatico.

3.6 Sprint Retrospective

Cosa ha funzionato

- ✓ **Integrazione GPX → altimetria → modello CAI:** La collaborazione tra Marco (modello Location GeoJSON + meteo TINIA) e Giacomo (parser GPX con smoothing + Canvas altimetrico) si è incastrata efficacemente, producendo la feature più innovativa dello sprint.
- ✓ **Refactor backend populate simmetrico:** Il fix di creatorId + participants.userId con Response<ApiMessageBody> ha sbloccato l'edit mode PATCH che crashava per deserializzazione Gson (bug già risolto durante lo sprint).
- ✓ **SessionStartCoordinator bus singleton:** L'introduzione del coordinator ha eliminato il coupling diretto tra SessionDetailScreen e RegistraScreen, rendendo il flusso AVVIA → GPS tracking pulito e disaccoppiato.

- ✓ **Dark theme Material3 coerente:** Token di design condivisi (TsmPrimary #3F7020, TsmAccent #4FC3F7, TsmSos #6B0D0D) hanno garantito coerenza visiva senza overhead.
- ✓ **Strategia di branching:** L'adozione di feature branches per ogni issue ha prevenuto conflitti accidentali e mantenuto main stabile per tutta la durata dello sprint.

Cosa non ha funzionato – Bug Critici identificati

3 Bug Critici – audit interno fine sprint

Bug C1 – Partecipante non-creator non può AVVIA: Il service `hikeSessionService.js` (`updateSessionStatus`) blocca il PATCH `/sessions/:id/status` con 403 FORBIDDEN se `creatorId ≠ userId`. Il client mostra il pulsante AVVIA a tutti i partecipanti, ma in `RegistraViewModel.autoStartFromSession()` il 403 è catturato silenziosamente da `runCatching`: il tracking GPS parte localmente, ma la sessione lato server resta in stato PLANNED (`startTime` mai popolato) → metriche server inconsistenti. Contraddice il flusso `SessionStartCoordinator` descritto nell'architettura. ✓ *Fix applicato 17/05: gating UI del pulsante AVVIA al solo creator in `SessionDetailScreen`; i partecipanti vedono un chip "In attesa del Capogruppo" finché lo status sessione è PLANNED, poi un bottone "UNISCITI AL TRACKING" quando lo status diventa ACTIVE.*

Bug C2 – Endpoint Weather non protetti: Il router `weatherRoutes.js` non chiamava `authenticate`. POST `/weather/seed` e POST `/forecast/:id/refresh` (operazioni pesanti verso API TINIA) erano pubblici → possibile DoS economico e blacklist IP del server. ✓ *Fix applicato 17/05: `authenticate + requireRoles("admin")` aggiunti; endpoint debug GET `/weather/test` rimosso.*

Bug C3 – ACCESS_BACKGROUND_LOCATION mancante: Il permesso non era dichiarato nell'AndroidManifest. Su Android 10+, `FusedLocationProvider` interrompe gli update GPS quando lo schermo si spegne → tracciato GPS troncato → metriche e punti CAI errati. Viola RNF9 ("tracking continuo background"). ✓ *Fix applicato 17/05: ACCESS_BACKGROUND_LOCATION e WAKE_LOCK aggiunti al manifest. Flusso runtime Android 10+ pianificato in Sprint 2.*

Debito tecnico noto (non bloccante)

Voci marcate ✓ sono state risolte durante l'audit di fine sprint del 17/05; voci marcate ○ restano debito aperto per Sprint 2.

- **Pattern Repository violato in 4 ViewModel:** `ActivityListViewModel`, `SessionPlanViewModel`, `SessionJoinViewModel` e `SessionDetailViewModel` chiamano `TsmApiClient.service()` direttamente, bypassando il layer Repository. Impatta testabilità e manutenibilità; refactor pianificato in Sprint 2 (1 ViewModel per PR).
- ✓ **User.sessionRoles non nello schema Mongoose** (risolto 17/05): il service faceva `$push` su un campo non dichiarato → Mongoose con `strict: true` scartava il write silenziosamente. *Fix:* aggiunto sub-document `sessionRoles: [{groupId, role, createdBy}]` allo schema `user.js`.
- **meetingDate come String invece di Date:** l'ordinamento `sort({meetingDate: 1})` è lessicografico, non temporale. Instabile cross-client se il formato cambia. Migrazione (`Date + script di backfill`) pianificata in Sprint 2.
- ✓ **joinSession ritornava documento non-populated** (risolto 17/05): rischio di crash Gson lato client. *Fix:* aggiunto `session.populate([creatorId, participants.userId])` in `hikeSessionService.js`.
- ✓ **Default [0,0] su startPoint.coordinates** (risolto 17/05): inquinava l'indice `2dsphere` con Null Island. *Fix:* rimosso il default e applicato `sparse: true` sull'indice `2dsphere`.
- **Zero unit test scritti:** Sprint 2 introdurrà almeno una test class `JUnit` per i ViewModel e un test `Jest` per `hikeSessionService`.
- ✓ **Codice morto rimosso** (17/05): cancellati `AppRepository.kt` (interfaccia vuota), `LocalDataSource.kt` (singleton vuoto) ed endpoint debug GET `/weather/test` (esposto in produzione).

Action items per Sprint 2

1. **Triage bug critici** (✓ **completato 17/05 – audit interno**): C1 → gating AVVIA al creator; C2 → `authenticate + requireRoles("admin")` su `/weather/seed` e `/refresh`; C3 → `ACCESS_BACKGROUND_LOCATION` aggiunto al manifest.
2. **Pulizia codice morto** (✓ **completato 17/05**): cancellati `AppRepository.kt`, `LocalDataSource.kt` e l'endpoint `GET /weather/test`.
3. **Setup CI minimale**: GitHub Actions con build APK (`./gradlew assembleDebug`) + ESLint backend, per intercettare regressioni automaticamente.
4. **Aggiornare KDoc orfani**: commenti obsoleti in `TsmApplication.kt` e `TsmApiService.kt` identificati durante l'audit.
5. **Migrare `meetingDate` a `Date`**: script di backfill MongoDB per i documenti esistenti (ordinamento temporale vs lessicografico).

Dinamiche di team e pratiche Agile

Il team ha adottato un approccio *parzialmente asincrono* per il Daily Scrum (via prettamente messaggistica), con sync call vocale settimanale. La principale lezione appresa: **3 bug critici scoperti solo nell'audit di fine sprint** evidenziano la necessità di un *Definition of Done check* sistematico al termine di ogni settimana, non solo a chiusura sprint. Questa pratica verrà istituzionalizzata in Sprint 2 con una sync call bisettimanale dedicata a verifica e triage.

A - API implementate Sprint 1

Tutti gli endpoint sono documentati in Swagger (`swagger-output.json` nel repository; UI su `/api-docs`).

Metodo	Route	Auth	Descrizione
POST	<code>/auth/login</code>	No	Login con email/password; ritorna JWT.
GET	<code>/auth/verify/:token</code>	No	Verifica email → redirect deep link <code>tsm://</code> .
POST	<code>/auth/forgot-password</code>	No	Richiesta reset password (invio link via SMTP).
GET	<code>/auth/reset-password/:token</code>	No	Form HTML responsive reset password.
POST	<code>/auth/reset-password/:token</code>	No	Salva nuova password (accetta JSON o form).
POST	<code>/users</code>	No	Registrazione utente o rifugio (<code>rifugioDetails</code>).
GET	<code>/users/:id</code>	JWT	Profilo utente.
PUT	<code>/users/:id</code>	JWT+admin	Aggiorna dati utente.
DELETE	<code>/users/:id</code>	JWT+admin	Elimina utente.
POST	<code>/api/v1/sessions</code>	JWT	Crea sessione (GPX stats + <code>estimatedPoints</code> CAI).
GET	<code>/api/v1/sessions/my</code>	JWT	Le mie sessioni (populate creator + partecipanti).
continua...			

Metodo	Route	Auth	Descrizione
GET	/api/v1/sessions/stats?year=	JWT	Statistiche aggregate attività COMPLETED per anno (totale km, dislivello, punti, conteggio/difficoltà mensili).
GET	/api/v1/sessions/:id	JWT	Dettaglio sessione fully populated.
POST	/api/v1/sessions/join	JWT	Join con codice TSM-XXXX.
POST	/api/v1/sessions/:id/leave	JWT	Abbandona sessione (non-creator).
DELETE	/api/v1/sessions/:id	JWT (creator)	Elimina sessione e rimuove partecipanti.
PATCH	/api/v1/sessions/:id	JWT (creator)	Modifica dettagli (populate fix Gson-safe).
PATCH	/api/v1/sessions/:id/status	JWT (creator)	Ciclo vita: PLANNED→ACTIVE →COMPLETED.
GET	/weather/locations/nearby	No*	Stazioni meteo vicine per coordinate (2dsphere).
GET	/weather/locations/search	No*	Ricerca stazioni per nome (case-insensitive).
GET	/weather/forecast/:externalId	No*	Forecast 3h + 24h (cache MongoDB 1h).
POST	/weather/forecast/:externalId/refresh	JWT+admin	Forza refresh forecast. ✓ Fix C2 17/05
POST	/weather/seed	JWT+admin	Popola DB con towns + POI da API TINIA. ✓ Fix C2 17/05

Tabella 9: Endpoint API implementati nello Sprint 1. *Gli endpoint meteo pubblici sono stati protetti con authenticate obbligatorio in data 17/05 (Fix Bug C2).

Tutti gli endpoint POST /weather/seed e POST /weather/forecast/:id/refresh richiedono ora Bearer JWT + role=admin (fix Bug C2 applicato 17/05). I GET /weather/ rimangono pubblici (dati meteo non sensibili).*

B Utilizzo Intelligenza Artificiale nello Sviluppo

Per rispettare le tempistiche previste dal progetto, lo sviluppo del software è stato affiancato da strumenti di intelligenza artificiale generativa, utilizzati come supporto alla progettazione, alla stesura del codice e alla documentazione. Tale scelta non ha sostituito le decisioni architettoniche né la responsabilità del team sul prodotto finale: ogni output prodotto dall'AI è stato revisionato, integrato e validato manualmente, in modo da garantire coerenza con i requisiti, con gli standard adottati e con la qualità attesa del sistema.

C Mockup delle Schermate

C.1 Schermate di Autenticazione e Registrazione

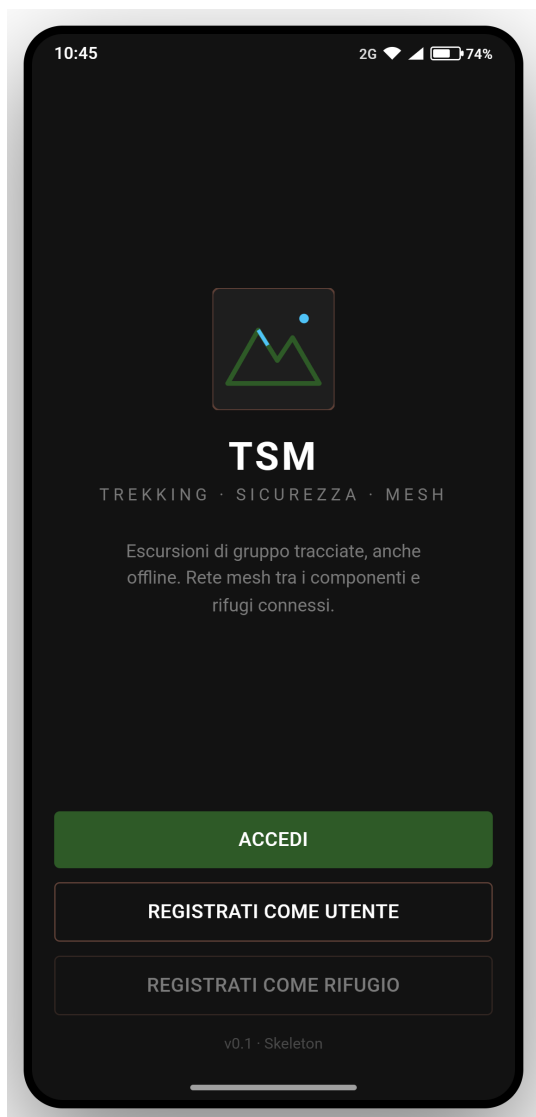


Figura 2: Welcome Screen

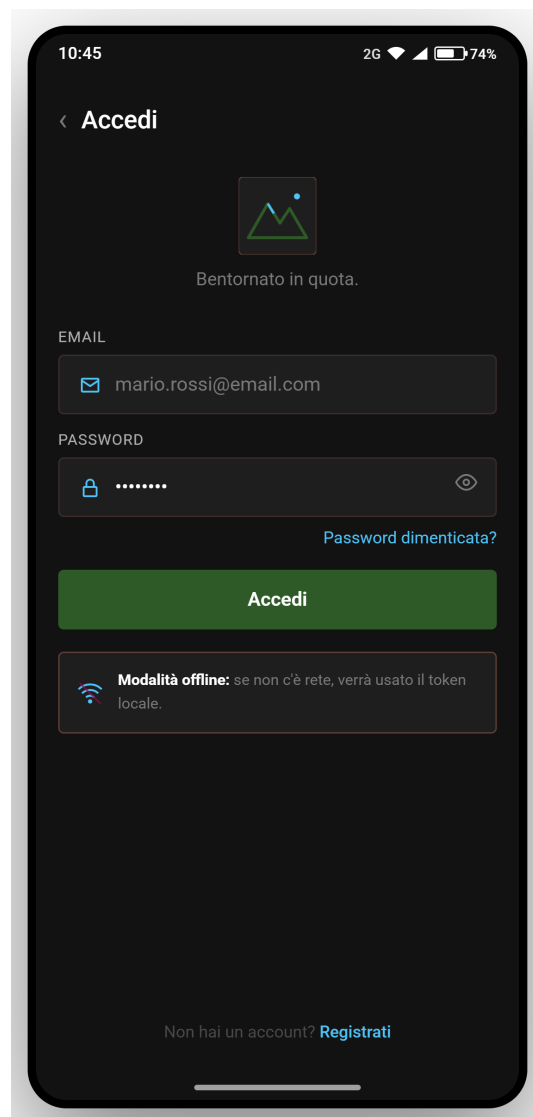
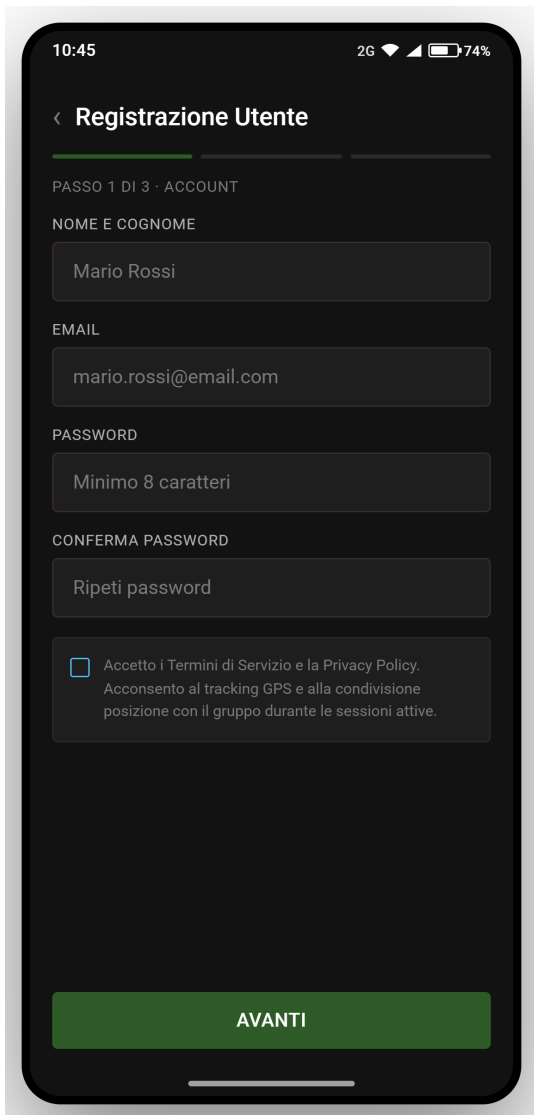


Figura 3: Login



10:45 2G 74%

< Registrazione Utente

PASSO 1 DI 3 - ACCOUNT

NOME E COGNOME

Mario Rossi

EMAIL

mario.rossi@email.com

PASSWORD

Minimo 8 caratteri

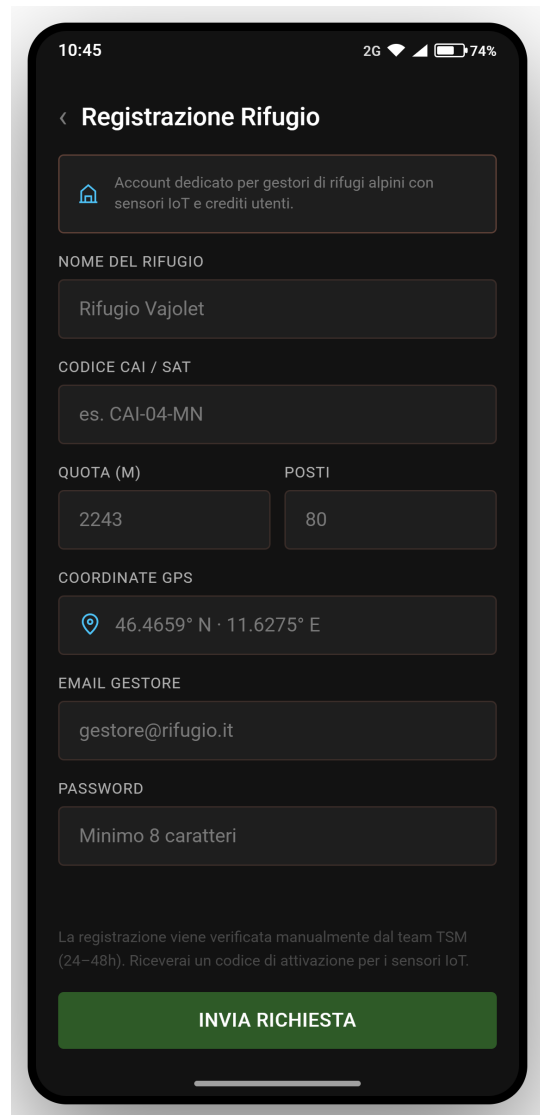
CONFERMA PASSWORD

Ripeti password

☐ Accetto i Termini di Servizio e la Privacy Policy. Acconsento al tracking GPS e alla condivisione posizione con il gruppo durante le sessioni attive.

AVANTI

Figura 4: Registrazione Utente



10:45 2G 74%

< Registrazione Rifugio

Account dedicato per gestori di rifugi alpini con sensori IoT e crediti utenti.

NOME DEL RIFUGIO

Rifugio Vajolet

CODICE CAI / SAT

es. CAI-04-MN

QUOTA (M)

2243

POSTI

80

COORDINATE GPS

46.4659° N · 11.6275° E

EMAIL GESTORE

gestore@rifugio.it

PASSWORD

Minimo 8 caratteri

La registrazione viene verificata manualmente dal team TSM (24-48h). Riceverai un codice di attivazione per i sensori IoT.

INVIA RICHIESTA

Figura 5: Registrazione Rifugio

C.2 Schermate Home e Attività

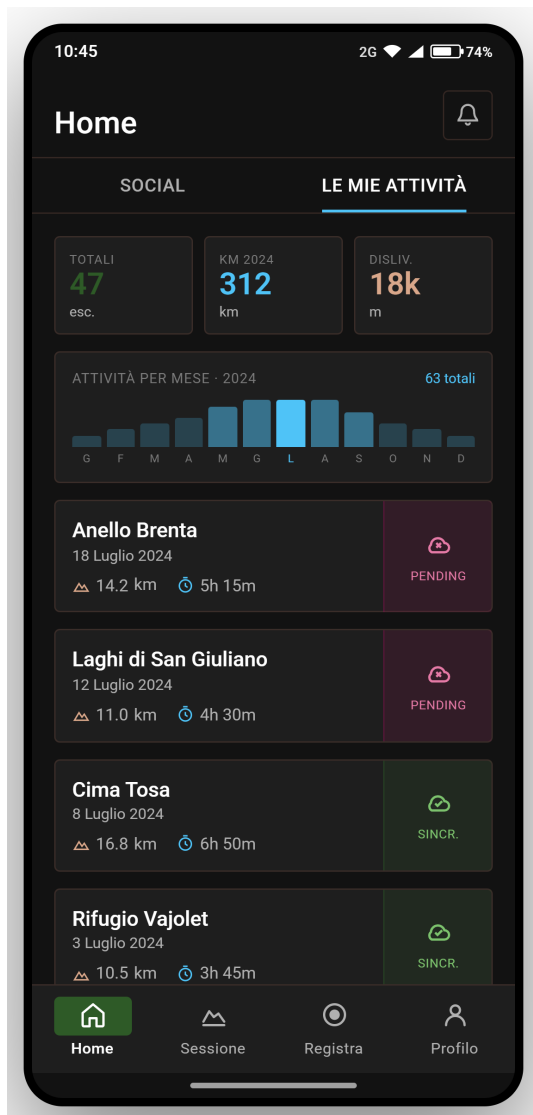


Figura 6: Home – Le Mie Attività



Figura 7: Dettaglio Attività



Figura 8: Le Mie Attività – Vista (1)

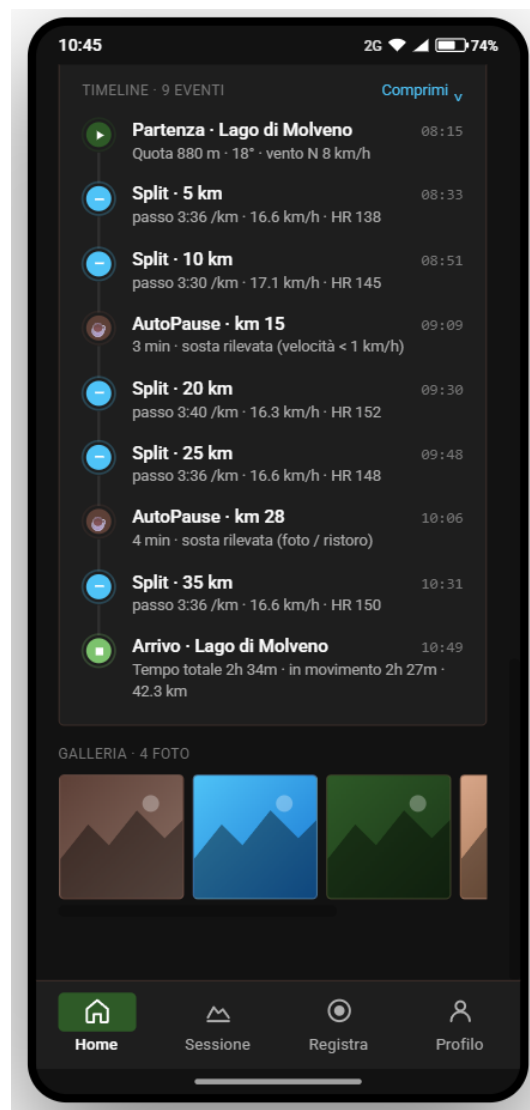


Figura 9: Le Mie Attività – Vista (2)

C.3 Schermate di Gestione Sessione Escursione

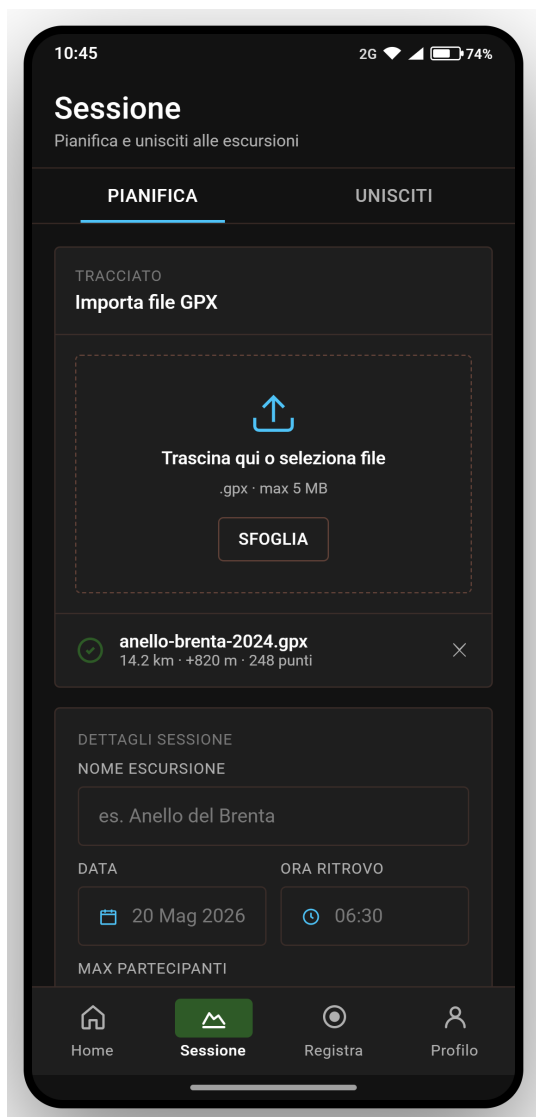


Figura 10: Sessione – Pianifica (1)

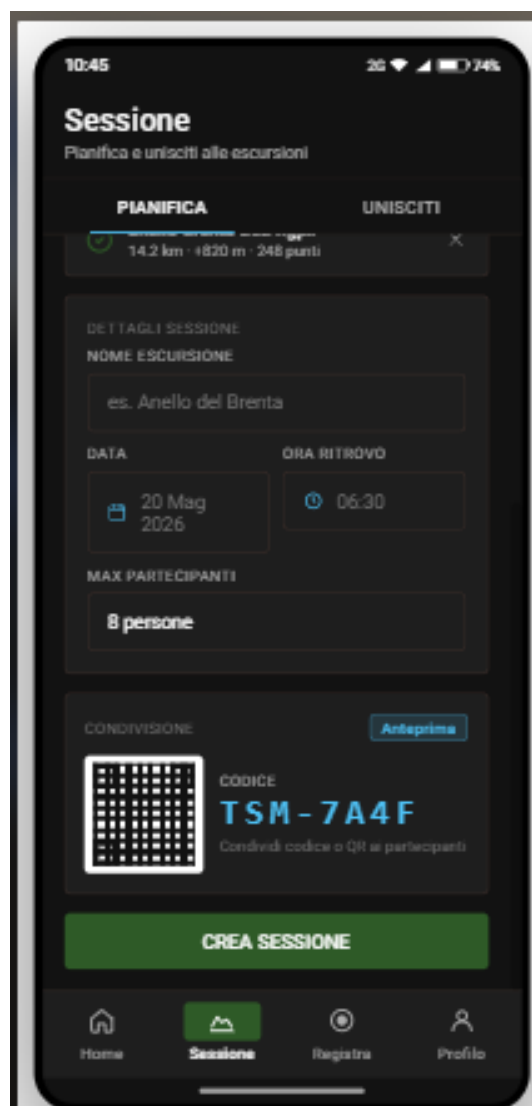


Figura 11: Sessione – Pianifica (2)

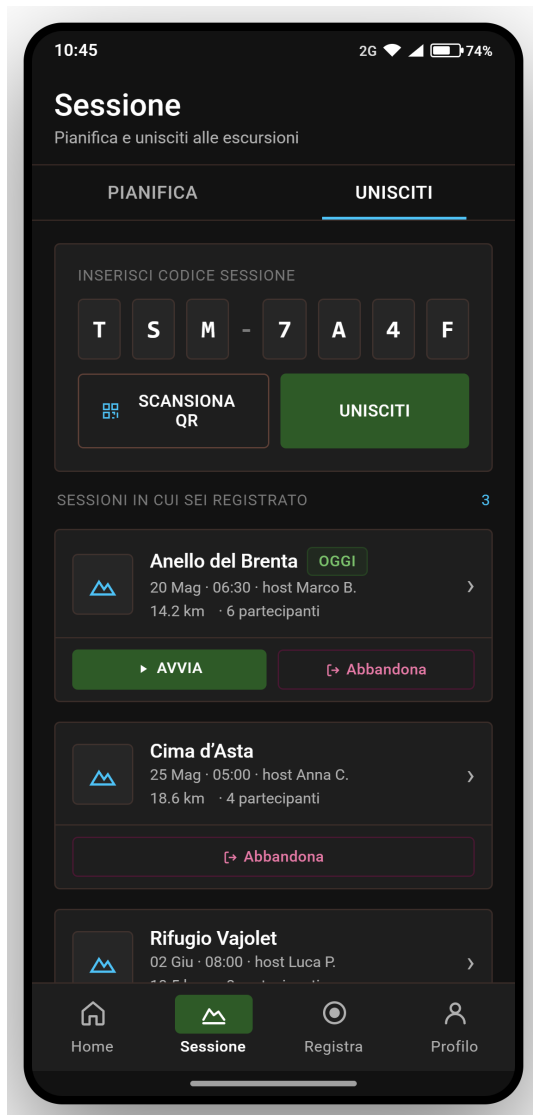


Figura 12: Sessione – Unisciti



Figura 13: Sessione – Dettaglio (1)

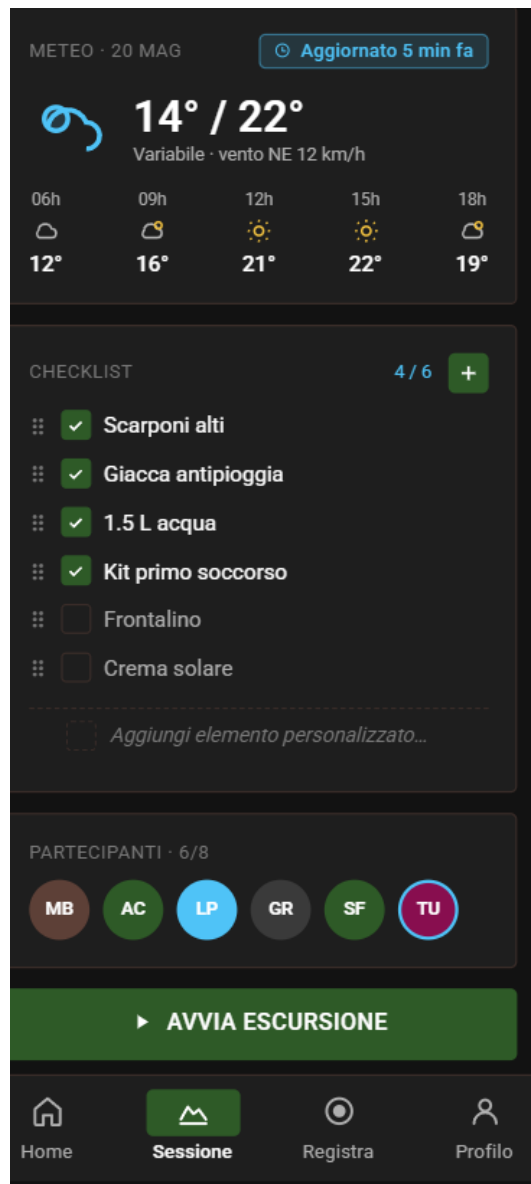


Figura 14: Sessione – Dettaglio (2)

C.4 Schermata di Tracciamento GPS

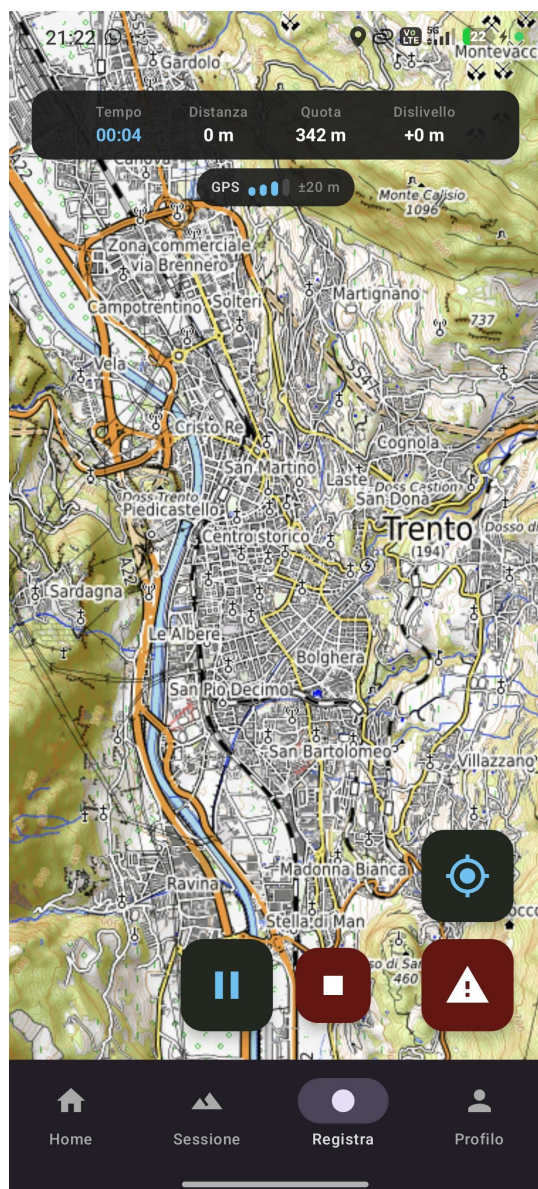


Figura 15: Registra – Tracking (1)



Figura 16: Registra – Tracking (2)